

Wstęp

W ramach ćwiczenia przygotowano dwa środowiska wirtualne: Kali Linux pełniący rolę stacji testowej oraz Ubuntu Server, na którym zainstalowano i skonfigurowano system Suricata. Następnie zweryfikowano konfigurację interfejsu sieciowego serwera oraz poprawność działania Suricata. Po przygotowaniu środowiska przystąpiono do analizy ruchu sieciowego w trybach Sniffer i Packet Logger oraz przechwytywania i rejestrowania pakietów sieciowych.

```
ubuntu@ubuntu-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:4a:26:da brd ff:ff:ff:ff:ff:ff
    altname enx0800274a26da
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 348sec preferred_lft 348sec
    inet6 fe80::a00:27ff:fe4a:26da/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-server:~$
```

Rysunek 1. Wykonanie komendy ip a

Przed rozpoczęciem ćwiczenia zapoznano się z podstawowymi trybami pracy systemu Suricata. Tryb Sniffer służy do przechwytywania i wyświetlania ruchu sieciowego w czasie rzeczywistym. Tryb Packet Logger umożliwia zapisywanie przechwyconych pakietów do plików PCAP w celu późniejszej analizy. Tryb NIDS (Network Intrusion Detection System) analizuje ruch sieciowy na podstawie zdefiniowanych reguł i generuje alerty w przypadku wykrycia potencjalnych zagrożeń. Dodatkowo Suricata może pracować w trybie IPS (Intrusion Prevention System), który oprócz wykrywania zagrożeń umożliwia również blokowanie ruchu sieciowego. Możliwa jest także analiza wcześniej zapisanych plików PCAP w trybie offline.

Podczas realizacji ćwiczenia korzystano z dokumentacji Suricata oraz informacji dostępnych w podręczniku systemowym man suricata, które zostały wykorzystane podczas konfiguracji i analizy działania poszczególnych trybów pracy.

Przed rozpoczęciem analizy zweryfikowano poprawność konfiguracji Suricata przy użyciu opcji -T. Narzędzie poprawnie załadowało plik konfiguracyjny i przeprowadziło jego test. Wystąpiło ostrzeżenie dotyczące braku pliku suricata.rules, ponieważ na tym etapie nie były jeszcze skonfigurowane reguły IDS. Nie miało to wpływu na działanie trybu Sniffer.

```
ubuntu@ubuntu-server:~$ suricata -h
This is Suricata version 8.0.3 RELEASE
Features: NFQ PCAP SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP NG LIBNET1.1
HAVE_HTTP_URI NORMALIZE_HOOK PCRE_JIT HAVE_NSS HTTP2_DECOMPRESSION HAVE_LUA HA
VE_JA3 HAVE_JA4 HAVE_LIBJANSSON TLS TLS_CRL MAGIC_RUST POPCNT64
SSE support: SSE_4_2 SSE_4_1 SSE_3 SSE_2
Atomic intrinsics: 1 2 4 8 16 byte(s)
64-bits, Little-endian architecture
GCC version 15.2.0, C version 201112
Compiled with _FORTIFY_SOURCE=2
L1 cache line size (CLS)=64
thread local storage method: _Thread_local
Compiled with LibHTP v8.0.3

Suricata Configuration:
AF_PACKET support: yes
AF_XDP support: yes
BPF support: yes
eBPF support: yes
XDP support: yes
PF_RING support: no
NFQueue support: yes
NetLOG support: yes
IPFW support: no
Netmap support: no
DAG enabled: no
Npattech enabled: no
WinDivert enabled: no
Npcap support: no

Unix socket enabled: yes
Detection enabled: yes

Libmagic support: yes
libjansson support: yes
hiredis support: yes
hiredis async with libevent: yes
PCRE jit: yes
GeoIP2 support: yes
JA3 support: yes
JA4 support: yes
Hyperscan support: yes
Hwloc support: yes
```

Rysunek 2. Suricata informacja konfiguracyjna

```

ubunt@ubuntserver:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
W: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
ubunt@ubuntserver:~$ ls -lah /var/lib/suricata/rules
total 8.0K
drwxr-xr-x 2 root root 4.0K Jan 27 18:55 .
drwxr-xr-x 4 root root 4.0K May 30 13:16 ..
ubunt@ubuntserver:~$ grep suricata.rules /etc/suricata/suricata.yaml
default-rule-path: /var/lib/suricata/rules
- suricata.rules

```

Rysunek 3. Weryfikacja konfiguracji Suricata

Tryb sniffer live

Następnie Suricata została uruchomiona na interfejsie sieciowym enp0s3 w celu przechwytywania pakietów. Ze względu na wykorzystanie Ubuntu Server bez środowiska graficznego ruch HTTP/HTTPS wygenerowano przy użyciu poleceń curl oraz ping. W logach zapisanych w pliku eve.json zarejestrowano różne typy komunikacji, między innymi DNS, ICMP, SSH, NTP oraz TLS/HTTPS. Dla każdego połączenia Suricata zapisała adres IP źródłowy i docelowy, numery portów, wykorzystany protokół oraz informacje o stanie połączenia.

Podczas testu przechwycono 358 pakietów, przy czym nie wystąpiła utrata żadnego z nich (0 drops). Potwierdza to poprawne działanie Suricata w trybie Sniffer oraz skuteczne monitorowanie aktywności sieciowej.

```

ubunt@ubuntserver:~$ sudo suricata -v -i enp0s3
Notice: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 2
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Warning: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
Warning: detect: 1 rule files specified, but no rules were loaded!
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 0 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 0 inspect application layer, 0 are decoder event only
Notice: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
Info: unix-manager: unix socket '/var/run/suricata/suricata-command.socket'
Info: runmodes: enp0s3: creating 2 threads
Info: ioctl: enp0s3: MTU 1500
Notice: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine started.

^CNotice: suricata: Signal Received. Stopping engine.
Info: suricata: time elapsed 193.480s
Info: counters: Alerts: 0
Notice: device: enp0s3: packets: 358, drops: 0 (0.00%), invalid chksum: 0
ubunt@ubuntserver:~$

```

Rysunek 4. Uruchomienie Suricata

Suricata została uruchomiona na interfejsie enp0s3 przy użyciu polecenia `suricata -v -i enp0s3`, co umożliwiło przechwytywanie całego ruchu sieciowego przechodzącego przez monitorowany interfejs.

```

ubunt@ubuntserver:~$ sudo tail -n 30 /var/log/suricata/eve.json
{"timestamp": "2026-05-31T08:51:48.477280+0000", "flow_id": 498143356220980, "in_iface": "enp0s3", "event_type": "flow", "src_ip": "10.0.2.15", "src_port": 35436, "dest_ip": "185.15.59.224", "dest_port": 443, "ip_v": 4, "proto": "TCP", "app_proto": "tls", "flow": {"pkts_toserver": 18, "pkts_toclient": 14, "bytes_toserver": 2905, "bytes_toclient": 7186, "start": "2026-05-31T08:51:13.771343+0000", "end": "2026-05-31T08:51:13.942832+0000", "age": 0, "state": "closed", "reason": "shutdown", "alerted": false, "tx_cnt": 1, "tcp": {"tcp_flags": "1f", "tcp_flags_ts": "1f", "tcp_flags_tc": "1b", "syn": true, "fin": true, "rst": true, "psh": true, "ack": true, "state": "closed", "ts_max_regions": 1, "tc_max_regions": 1}}
{"timestamp": "2026-05-31T08:51:48.488307+0000", "flow_id": 2150406815547764, "in_iface": "enp0s3", "event_type": "flow", "src_ip": "10.0.2.15", "src_port": 37378, "dest_ip":

```

:"194.146.251.102","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:48:47.173000+0000","end":"2026-05-31T08:48:47.203412+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.488936+0000","flow_id":724030663819649,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":36234,"dest_ip":"51.38.131.245","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:49:54.496256+0000","end":"2026-05-31T08:49:54.525049+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.489333+0000","flow_id":1179434699066367,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":37623,"dest_ip":"194.146.251.114","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:49:48.864432+0000","end":"2026-05-31T08:49:48.994736+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.489371+0000","flow_id":1078920276746485,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":55458,"dest_ip":"51.68.141.5","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:50:51.054597+0000","end":"2026-05-31T08:50:51.082709+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.489405+0000","flow_id":603387708514678,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":58697,"dest_ip":"162.159.200.123","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:50:58.730311+0000","end":"2026-05-31T08:50:58.759403+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.489438+0000","flow_id":985003092420534,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":41946,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":81,"bytes_toclient":109,"start":"2026-05-31T08:50:03.950234+0000","end":"2026-05-31T08:50:03.980730+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-31T08:51:48.489696+0000","flow_id":1106960545695537,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":36675,"dest_ip":"156.17.245.123","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:49:55.126662+0000","end":"2026-05-31T08:49:55.170793+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.489736+0000","flow_id":1437305188342180,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":60829,"dest_ip":"193.25.222.136","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:50:53.924472+0000","end":"2026-05-31T08:50:53.985403+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.490005+0000","flow_id":471553729518714,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":50403,"dest_ip":"94.154.96.7","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:48:49.634080+0000","end":"2026-05-31T08:48:49.671871+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.490180+0000","flow_id":1785788239997436,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":49257,"dest_ip":"89.161.47.136","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:48:54.874538+0000","end":"2026-05-31T08:48:55.035753+0000","age":1,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.490212+0000","flow_id":346605682193017,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":41142,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":84,"bytes_toclient":112,"start":"2026-05-31T08:51:13.736060+0000","end":"2026-05-31T08:51:13.769920+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-31T08:51:48.490375+0000","flow_id":982880927053175,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":48097,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":81,"bytes_toclient":177,"start":"2026-05-31T08:50:03.949740+0000","end":"2026-05-31T08:50:03.974032+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-31T08:51:48.490414+0000","flow_id":2181050522043503,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":37744,"dest_ip":"89.161.47.136","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:51:03.966567+0000","end":"2026-05-31T08:51:04.348180+0000","age":1,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.490588+0000","flow_id":287146480016460,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":57841,"dest_ip":"212.132.97.26","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T08:49:53.525608+0000","end":"2026-05-31T08:49:53.585424+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T08:51:48.490615+0000","flow_id":1124183378861909,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":142.250.109.102,"ip_v":4,"proto":"ICMP","icmp_code":8,"icmp_code":0,"response_icmp_type":0,"response_icmp_code":0,"flow":{"pkts_toserver":29,"pkts_toclient":29,"bytes_toserver":2842,"bytes_toclient":2842,"start":"2026-05-31T08:50:03.982640+0000","end":"2026-05-31T08:50:34.310746+0000","age":31,"state":"established","reason":"shutdown","alerted":false}}
{"timestamp":"2026-05-31T08:51:48.490800+0000","flow_id":344644665123942,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":38278,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":84,"bytes_toclient":109,"start":"2026-05-31T08:50:03.950234+0000","end":"2026-05-31T08:50:03.980730+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}

```

client":100,"start":"2026-05-31T08:51:13.735603+0000","end":"2026-05-
31T08:51:13.768113+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-
31T08:51:48.491064+0000","flow_id":1945476948996419,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.2","src_port":64581,"dest_ip":
"10.0.2.15","dest_port":22,"ip_v":4,"proto":"TCP","flow":{"pkts_toserver":2,"pkts_toclient":1,"bytes_toserver":178,"bytes_toclient":118,"start":"2
026-05-31T08:51:34.518502+0000","end":"2026-05-
31T08:51:34.687064+0000","age":0,"state":"new","reason":"shutdown","alerted":false,"exception_policy":{"target":"stream_midstream","policy":
"ignore"}},"tcp":{"tcp_flags":"00","tcp_flags_ts":"00","tcp_flags_tc":"00"}}
{"timestamp":"2026-05-
31T08:51:48.491277+0000","flow_id":230493765874734,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":33687,"dest_ip":
"162.159.200.123","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"byt
es_toclient":90,"start":"2026-05-31T08:48:48.774562+0000","end":"2026-05-
31T08:48:48.803163+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-
31T08:51:48.491318+0000","flow_id":164845996653240,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.2","src_port":56558,"dest_ip":
"10.0.2.15","dest_port":22,"ip_v":4,"proto":"TCP","app_proto":"ssh","flow":{"pkts_toserver":83,"pkts_toclient":85,"bytes_toserver":10004,"bytes_t
oclient":18828,"start":"2026-05-31T08:49:12.234989+0000","end":"2026-05-
31T08:51:45.620426+0000","age":153,"state":"closed","reason":"shutdown","alerted":false,"tx_cnt":1},"tcp":{"tcp_flags":"1b","tcp_flags_ts":"1b",
"tcp_flags_tc":"1b","syn":true,"fin":true,"psh":true,"ack":true,"state":"closed","ts_max_regions":1,"tc_max_regions":1}}
{"timestamp":"2026-05-
31T08:51:48.491500+0000","flow_id":831536568322993,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":35975,"dest_ip":
"51.68.141.5","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_t
oclient":90,"start":"2026-05-31T08:48:42.521287+0000","end":"2026-05-
31T08:48:42.551692+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}

```

Logi 1. Suricata w trybie sniffer live

Przechwycone logi przedstawiają rzeczywisty ruch sieciowy generowany przez system Ubuntu podczas wykonywania testów. W logach widoczne są połączenia DNS realizowane na porcie 53, których zadaniem było tłumaczenie nazw domenowych na adresy IP. Zarejestrowano również ruch ICMP związany z poleceniem ping wykorzystywanym do sprawdzenia dostępności hostów sieciowych.

W logach występują także połączenia SSH na porcie 22, odpowiadające zdalnemu zarządzaniu serwerem Ubuntu. Dodatkowo Suricata rozpoznała ruch TLS na porcie 443, czyli szyfrowane połączenia HTTPS wykorzystywane podczas komunikacji z usługami internetowymi. Widoczne są również pakiety NTP na porcie 123 odpowiedzialne za synchronizację czasu systemowego.

Na podstawie logów można odczytać adresy IP źródłowe i docelowe, wykorzystywane protokoły, numery portów, czas wystąpienia zdarzeń oraz informacje o stanie połączeń sieciowych. Przechwycone dane potwierdzają poprawne działanie Suricata w trybie Sniffer oraz skuteczne monitorowanie ruchu sieciowego w czasie rzeczywistym.

Co zawierają logi?

Logi Suricata zawierają informacje o czasie zdarzenia, adresach IP źródłowych i docelowych, numerach portów, użytych protokołach, liczbie przesłanych pakietów i bajtów, stanie połączenia oraz rozpoznany protokole aplikacyjnym (np. DNS, SSH, TLS, NTP). W przypadku wykrycia zagrożenia log może również zawierać identyfikator reguły, priorytet oraz opis alertu.

Jak można skonfigurować logi Suricata?

Konfiguracja logowania znajduje się w pliku `/etc/suricata/suricata.yaml`. Można określić format logów (eve.json, fast.log, stats.log), zakres zapisywanych informacji, poziom szczegółowości, rodzaj monitorowanych zdarzeń oraz lokalizację plików logów. Suricata umożliwia również zapis logów w formacie JSON, Syslog oraz PCAP.

Co będzie w logach?

Zawartość logów zależy od wybranego trybu pracy i konfiguracji. Mogą one zawierać informacje o przechwyconych pakietach, przepływach sieciowych (flows), sesjach TCP/UDP, zapytaniach DNS, połączeniach HTTP/HTTPS, błędach protokołów oraz alertach wygenerowanych przez reguły IDS.

Czy logi zawierają treść pakietów?

Tak, Suricata może analizować zawartość pakietów, jednak zależy to od konfiguracji logowania i użytych reguł. W przypadku nieszyfrowanych protokołów, takich jak HTTP, FTP czy Telnet, możliwa jest analiza przesyłanej treści. Dla protokołów szyfrowanych, takich jak HTTPS, widoczne są głównie metadane połączenia.

Czy można filtrować po treści przesyłanej do odbiorcy?

Tak. Suricata umożliwia tworzenie reguł wykorzystujących słowo kluczowe content, dzięki czemu można wyszukiwać określone ciągi znaków w ruchu sieciowym. Pozwala to wykrywać np. nazwy użytkowników, hasła, charakterystyczne fragmenty zapytań HTTP lub znane sygnatury ataków.

Czy wszystkie protokoły mogą być analizowane?

Nie wszystkie protokoły mogą być analizowane w takim samym stopniu. Suricata bardzo dobrze obsługuje protokoły TCP, UDP, HTTP, DNS, FTP, SMTP, SSH oraz TLS. W przypadku ruchu szyfrowanego możliwa jest analiza metadanych połączenia, natomiast pełna analiza treści wymaga odszyfrowania ruchu. Niektóre niestandardowe protokoły mogą być rozpoznawane jedynie częściowo.

Tryb packet logger

W celu uruchomienia trybu Packet Logger włączono moduł pcap-log w pliku suricata.yaml. Moduł ten umożliwia zapisywanie przechwyconych pakietów do plików w formacie PCAP, które mogą być później analizowane za pomocą narzędzi takich jak Wireshark lub tcpdump. Po uruchomieniu Suricata oraz wygenerowaniu ruchu sieciowego został utworzony plik log.pcap.1780219031, zawierający przechwycone pakiety sieciowe. Potwierdza to poprawne działanie trybu Packet Logger.

```
ubunt@ubuntserver:~$ sudo nano /etc/suricata/suricata.yaml
ubunt@ubuntserver:~$ grep -n pcap-log -A 15 /etc/suricata/suricata.yaml
422: - pcap-log:
423:   enabled: yes
424:   filename: log.pcap
425:
426:   # File size limit. Can be specified in kb, mb, gb. Just a number
427:   # is parsed as bytes.
428:   limit: 1000 MiB
429:
430:   # If set to a value, ring buffer mode is enabled. Will keep maximum
431:   # of
432:   # "max-files" of size "limit"
433:   max-files: 2000
434:
435:   # Compression algorithm for pcap files. Possible values: none, lz4.
436:   # Note also that on Windows, enabling compression will *increase* disk I/O.
437:   compression: none
438:
439: --
440: 2017: pcap-log:
441:   enabled: no
442:   filename: pcaplog_stats.log
443:   append: yes
444:
445: 2022-##
446: 2023-## Netfilter integration
447: 2024-##
448: 2025-
449: 2026-# When running in NFQ inline mode, it is possible to use a simulated
450: 2027-# non-terminal NFQUEUE verdict.
451: 2028-# This permits sending all needed packet to Suricata via this rule:
452: 2029-#     iptables -I FORWARD -m mark ! --mark $MARK/$MASK -j NFQUEUE
453: 2030-# And below, you can have your standard filtering ruleset. To activate
454: 2031-# this mode, you need to set mode to 'repeat'
455: 2032-# If you want a packet to be sent to another queue after an ACCEPT decision
456:
457: ubunt@ubuntserver:~$
```

Rysunek 5. Przedstawienie moduła pcap-log

```
ubunt@ubuntserver:~$ sudo suricata -i enp0s3
i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
W: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
W: detect: 1 rule files specified, but no rules were loaded!
i: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
i: log-pcap: Ring buffer initialized with 0 files.
i: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine started.
^Ci: suricata: Signal Received. Stopping engine.
i: device: enp0s3: packets: 195, drops: 0 (0.00%), invalid checksum: 0
ubunt@ubuntserver:~$ find /var/log/suricata -name "*.pcap"
ubunt@ubuntserver:~$ ls -lah /var/log/suricata
total 552K
drwxr-xr-x  2 root root  4.0K May 31 09:17 .
drwxrwxr-x 14 root syslog 4.0K May 30 13:16 ..
-rw-r--r--  1 root root 302K May 31 09:17 eve.json
-rw-r--r--  1 root root   0 May 30 13:16 fast.log
-rw-r--r--  1 root root  48K May 31 09:17 log.pcap.1780219031
-rw-r--r--  1 root root 164K May 31 09:17 stats.log
-rw-r--r--  1 root root  27K May 31 09:17 suricata.log
ubunt@ubuntserver:~$
```

Rysunek 6. Uruchomienie Suricata i przedstawienie pliku log.pcap.1780219031

```

ubunt@ubuntserver:~$ sudo cp /var/log/suricata/log.pcap.1780219031 /tmp/test.pcap
ubunt@ubuntserver:~$ sudo tcpdump -nn -r /tmp/test.pcap | head -20
reading from file /tmp/test.pcap, link-type EN10MB (Ethernet), snapshot length 262144
09:17:11.805202 IP 10.0.2.15.22 > 10.0.2.2.64581: Flags [P.], seq 4029834843:4029835003, ack 77042, win
65535, length 160
09:17:11.922994 IP 10.0.2.2.64581 > 10.0.2.15.22: Flags [.], ack 160, win 32464, length 0
09:17:15.080825 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [P.], seq 74319:74399, ack 3235178473, win 31776,
length 80
09:17:15.081798 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 1:113, ack 80, win 65535, length 112
09:17:15.193004 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [.], ack 113, win 31664, length 0
09:17:25.252874 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [P.], seq 80:288, ack 113, win 31664, length 208
09:17:25.253701 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 113:193, ack 288, win 65535, length 80
09:17:25.286578 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [.], ack 193, win 31584, length 0
09:17:26.093873 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [P.], seq 288:352, ack 193, win 31584, length 64
09:17:26.094846 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 193:305, ack 352, win 65535, length 112
09:17:26.104090 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [.], ack 305, win 31472, length 0
09:17:26.114122 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 305:593, ack 352, win 65535, length 288
09:17:26.118459 IP 10.0.2.15.40948 > 192.168.0.1.53: 5193+ [1au] A? google.com. (39)
09:17:26.119215 IP 10.0.2.15.52665 > 192.168.0.1.53: 34822+ [1au] AAAA? google.com. (39)
09:17:26.149144 IP 192.168.0.1.53 > 10.0.2.15.52665: 34822 4/0/1 AAAA 2a00:1450:4025:807::64, AAAA
2a00:1450:4025:807::65, AAAA 2a00:1450:4025:807::71, AAAA 2a00:1450:4025:807::8b (151)
09:17:26.148215 IP 192.168.0.1.53 > 10.0.2.15.40948: 5193 6/0/1 A 142.250.109.102, A 142.250.109.113, A
142.250.109.138, A 142.250.109.139, A 142.250.109.100, A 142.250.109.101 (135)
09:17:26.151296 IP 10.0.2.15 > 142.250.109.102: ICMP echo request, id 52812, seq 1, length 64
09:17:26.151526 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 593:705, ack 352, win 65535, length 112
09:17:26.152191 IP 10.0.2.2.56611 > 10.0.2.15.22: Flags [.], ack 705, win 32768, length 0
09:17:26.178070 IP 10.0.2.15.22 > 10.0.2.2.56611: Flags [P.], seq 705:849, ack 352, win 65535, length 144
tcpdump: Unable to write output: Broken pipe
ubunt@ubuntserver:~$

```

Logi 2. Suricata w trybie Packet Logger

Przedstawiony fragment pochodzi z pliku PCAP utworzonego przez Suricata w trybie Packet Logger i odczytanego za pomocą programu tcpdump. W logach widoczne są pakiety SSH przesyłane pomiędzy stacją zarządzającą a serwerem Ubuntu, zapytania DNS dotyczące domeny google.com oraz odpowiedzi serwera DNS zawierające odpowiadające adresy IPv4 i IPv6. Zarejestrowano również pakiety ICMP wygenerowane przez polecenie ping.

Logi pozwalają prześledzić kolejność komunikacji pomiędzy hostami, analizować adresy IP, porty źródłowe i docelowe, długości pakietów oraz wykorzystywane protokoły. W przeciwieństwie do trybu Sniffer, dane zostały zapisane do pliku PCAP, co umożliwia ich późniejszą analizę przy użyciu narzędzi takich jak tcpdump lub Wireshark.

Jakich informacji dostarcza Suricata w trybie Sniffer?

Tryb Sniffer umożliwia przechwytywanie i analizowanie ruchu sieciowego w czasie rzeczywistym. Na podstawie uzyskanych logów można odczytać adresy IP źródłowe i docelowe, numery portów, wykorzystywane protokoły, znaczniki czasu oraz informacje o bieżących połączeniach sieciowych.

Jakich informacji dostarcza Suricata w trybie Packet Logger?

Tryb Packet Logger zapisuje przechwycony ruch do plików PCAP. Na podstawie zapisanych pakietów można przeprowadzić szczegółową analizę komunikacji sieciowej, odtworzyć przebieg połączeń oraz analizować przesyłane dane przy użyciu narzędzi takich jak Wireshark lub tcpdump.

Jak można ustawić reguły i progi detekcji?

Reguły detekcji definiowane są w plikach reguł Suricata, np. local.rules. Każda reguła określa warunki wykrywania określonego zdarzenia, takie jak adres IP, port, protokół lub zawartość pakietu. Progi detekcji

można konfigurować za pomocą mechanizmów `threshold` oraz `detection_filter`, które określają liczbę zdarzeń wymaganą do wygenerowania alertu.

Co można odczytać z wyników działania Suricata w trybie Sniffer?

Na podstawie uzyskanych logów można odczytać adresy IP hostów uczestniczących w komunikacji, numery portów, wykorzystane protokoły sieciowe oraz czas wystąpienia zdarzeń. W przeprowadzonym ćwiczeniu zarejestrowano między innymi ruch DNS, ICMP, SSH, NTP oraz TLS/HTTPS. Logi pozwoliły określić kierunek komunikacji, typ połączenia oraz podstawowe informacje o przepływie danych pomiędzy hostami.

Co można odczytać z wyników działania Suricata w trybie Packet Logger?

Na podstawie zapisanych pakietów można przeanalizować szczegółowy przebieg komunikacji sieciowej. W przechwyconym pliku PCAP widoczne były pakiety SSH pomiędzy stacją zarządzającą a serwerem Ubuntu, zapytania DNS dotyczące domeny `google.com`, odpowiedzi serwera DNS zawierające adresy IPv4 i IPv6 oraz pakiety ICMP generowane przez polecenie `ping`. Dodatkowo możliwe było odczytanie adresów IP, portów źródłowych i docelowych, długości pakietów oraz kolejności wymiany danych pomiędzy hostami.

Różnice między trybami

Tryb Sniffer służy do monitorowania ruchu sieciowego w czasie rzeczywistym i prezentowania informacji na bieżąco. Tryb Packet Logger zapisuje przechwycony ruch do plików PCAP przeznaczonych do późniejszej analizy. Sniffer sprawdza się podczas bieżącej obserwacji aktywności sieciowej, natomiast Packet Logger jest szczególnie przydatny podczas analizy incydentów, archiwizacji ruchu oraz szczegółowej analizy pakietów.

Reguły akcji w systemie Suricata

Czy te reguły odpowiadają regułom firewalli z poprzedniego ćwiczenia?

Częściowo tak. Reguły `pass`, `drop` oraz `reject` działają podobnie do reguł stosowanych w firewallach `iptables` i `nftables`, ponieważ pozwalają odpowiednio przepuszczać lub blokować ruch sieciowy. Pozostałe reguły są związane z monitorowaniem i wykrywaniem zdarzeń.

Jakie są dodatkowe reguły, których nie było w `nftables/iptables`?

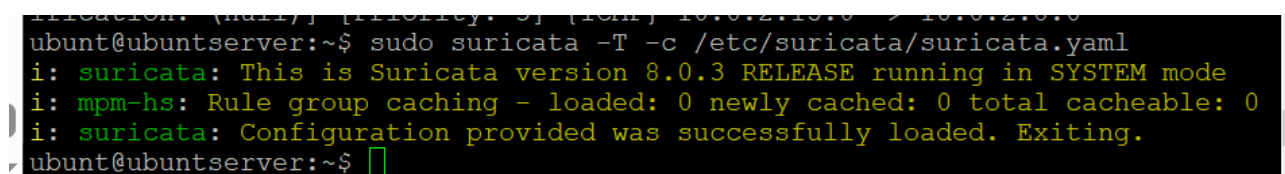
Dodatkowymi regułami są `alert`, `log` oraz `sdrop`. Reguła `alert` generuje alarm o wykrytym zdarzeniu, `log` zapisuje informacje o ruchu do logów, natomiast `sdrop` odrzuca pakiet bez generowania informacji o tym zdarzeniu. Reguły te są charakterystyczne dla systemów IDS/IPS i nie występują w klasycznych firewallach w takiej postaci.

Która reguła dotyczy systemu IDS?

Regułą najbardziej charakterystyczną dla systemu IDS jest `alert`. Jej zadaniem jest wykrywanie podejrzanych zdarzeń i generowanie alertów bez blokowania ruchu sieciowego. Dzięki temu administrator może zostać poinformowany o potencjalnym zagrożeniu przy zachowaniu normalnego działania sieci.

Konfiguracja trybu NIDS

W celu sprawdzenia poprawności działania trybu NIDS utworzono własną regułę wykrywającą pakiety ICMP. Następnie uruchomiono Suricata i wygenerowano ruch sieciowy za pomocą polecenia `ping`. W logach systemu pojawił się alert zgodny z utworzoną regułą, co potwierdziło poprawne działanie mechanizmu detekcji oraz poprawne ładowanie pliku `local.rules`.



```
ubuntu@ubuntu-server:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
i: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
i: suricata: Configuration provided was successfully loaded. Exiting.
ubuntu@ubuntu-server:~$
```

Rysunek 7. Sprawdzenie załadowania reguł


```

ubunt@ubuntserver:~$ sudo tail -20 /var/log/suricata/fast.log
05/31/2026-11:23:43.968373  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.8:8 -> 10.0.2.15:0
05/31/2026-11:23:43.968409  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.15:0 -> 10.0.2.8:0
ubunt@ubuntserver:~$

```

Rysunek 8. Sprawdzenie logów po testach

Po potwierdzeniu poprawnego działania mechanizmu detekcji utworzono zestaw własnych reguł NIDS przeznaczonych do wykrywania wybranych zagrożeń sieciowych. Reguły zostały zapisane w pliku `local.rules` i obejmowały między innymi detekcję skanowania portów, prób ataku SYN Flood, prób logowania użytkowników uprzywilejowanych przez usługę SSH, skanowania portów 8888–8890, nietypowych kombinacji flag TCP, wykorzystania nieprawidłowych metod HTTP oraz prób wstrzykiwania kodu JavaScript.

Po zapisaniu reguł przeprowadzono weryfikację konfiguracji przy użyciu polecenia:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Suricata poprawnie załadowała wszystkie reguły i zakończyła test bez błędów konfiguracji. Pojawiło się jedynie ostrzeżenie dotyczące jednej z reguł monitorujących porty 8888–8890, które nie wpływało na działanie systemu ani proces detekcji.

```

ubunt@ubuntserver:~$ sudo cat /var/lib/suricata/rules/local.rules
# a) Detekcja skanowania portów
alert tcp any any -> $HOME_NET any (msg:"PORT SCAN DETECTED"; flags:S; thresh
old:type both, track by_src, count 10, seconds 5; sid:1000001; rev:1;)

# d) Detekcja SYN flood
alert tcp any any -> $HOME_NET any (msg:"SYN FLOOD DETECTED"; flags:S; thresh
old:type both, track by_src, count 20, seconds 10; sid:1000002; rev:1;)

# f) Detekcja prob logowania uzytkownika root przez SSH
alert tcp any any -> $HOME_NET [22,1022,1122,1222,1322,2222,4022,8022] (msg:"
SSH ROOT LOGIN ATTEMPT"; content:"root"; nocase; sid:1000003; rev:1;)

# f) Detekcja prob logowania uzytkownika admin przez SSH
alert tcp any any -> $HOME_NET [22,1022,1122,1222,1322,2222,4022,8022] (msg:"
SSH ADMIN LOGIN ATTEMPT"; content:"admin"; nocase; sid:1000004; rev:1;)

# i) Detekcja prob polaczenia na porty 8888-8890
alert tcp any any -> $HOME_NET [8888,8889,8890] (msg:"HTTP TEST PORT 8888-889
0 DETECTED"; flags:S; sid:1000005; rev:1;)

# l) Detekcja flag TCP URG + PSH
alert tcp any any -> $HOME_NET any (msg:"TCP FLAGS URG PSH DETECTED"; flags:U
P; sid:1000006; rev:1;)

# l) Detekcja flag TCP URG + RST
alert tcp any any -> $HOME_NET any (msg:"TCP FLAGS URG RST DETECTED"; flags:U
R; sid:1000007; rev:1;)

# l) Detekcja flag TCP PSH + RST
alert tcp any any -> $HOME_NET any (msg:"TCP FLAGS PSH RST DETECTED"; flags:P
R; sid:1000008; rev:1;)

# o) Detekcja niewlasciwej metody HTTP TRACE
alert http any any -> $HOME_NET any (msg:"INVALID HTTP METHOD TRACE DETECTED"
; http.method; content:"TRACE"; sid:1000009; rev:1;)

# p) Detekcja wstrzykiwania kodu JavaScript
alert http any any -> $HOME_NET any (msg:"JAVASCRIPT INJECTION DETECTED"; htt
p.uri; content:"<script>"; nocase; sid:1000010; rev:1;)
ubunt@ubuntserver:~$

```

Rysunek 9. Konfiguracja własnych reguł NIDS

```

ubunt@ubuntserver:~$ sudo nano /var/lib/suricata/rules/local.rules
ubunt@ubuntserver:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
W: detect: rule 1000005: SYN-only to port(s) 8888:8888 w/o direction specifie
d, disabling for toclient direction
i: mpm-hs: Rule group caching - loaded: 0 newly cached: 3 total cacheable: 3
i: suricata: Configuration provided was successfully loaded. Exiting.
ubunt@ubuntserver:~$

```

Rysunek 10. Weryfikacja konfiguracji NIDS

Po poprawnym załadowaniu reguł uruchomiono Suricata w trybie NIDS i przeprowadzono testy z wykorzystaniem programu Nmap uruchomionego na maszynie Kali Linux. W celu sprawdzenia skuteczności detekcji wykonano skanowanie portów metodą SYN Scan, skanowanie portów 8888–8890 oraz skanowanie wykorzystujące niestandardowe flagi TCP.

Podczas testów wykorzystano następujące polecenia:

```
nmap -sS 10.0.2.15
```

```
nmap -sS -p 8888-8890 10.0.2.15
```

```
nmap -sX 10.0.2.15
```

```
nmap -sF 10.0.2.15
```

```
nmap -sN 10.0.2.15
```

Po wykonaniu testów przeanalizowano logi wygenerowane przez Suricata. System poprawnie wykrył przygotowane zdarzenia i wygenerował odpowiednie alerty zapisane w pliku fast.log.

```
(kali@kali)-[~]
$ nmap -sS 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:27 -0400
Nmap scan report for 10.0.2.15
Host is up (0.000096s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds

(kali@kali)-[~]
$ nmap -sS -p 8888-8890 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:27 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00088s latency).

PORT      STATE SERVICE
8888/tcp  closed sun-answerbook
8889/tcp  closed ddi-tcp-2
8890/tcp  closed ddi-tcp-3
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.58 seconds

(kali@kali)-[~]
$ nmap -sX 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:27 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00096s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
80/tcp    open|filtered http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.79 seconds
```

Rysunek 11. Wykonanie testów

```
(kali@kali)-[~]
$ nmap -sS -TS 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:32 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00078s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds

(kali@kali)-[~]
$ nmap -sF 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:32 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00095s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
80/tcp    open|filtered http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.79 seconds

(kali@kali)-[~]
$ nmap -sN 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 09:32 -0400
Nmap scan report for 10.0.2.15
Host is up (0.0014s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
80/tcp    open|filtered http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.79 seconds

(kali@kali)-[~]
$
```

Rysunek 12. Wykonanie testów

```

ubuntu@ubuntu-server:~$ sudo tail -20 /var/log/suricata/fast.log
[sudo: authenticate] Password:
05/31/2026-11:23:43.968373  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.8:8 -> 10.0.2.15:0
05/31/2026-11:23:43.968409  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.15:0 -> 10.0.2.8:0
05/31/2026-11:58:37.050484  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:143
05/31/2026-11:58:37.050485  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.
2.15:8888
05/31/2026-11:58:37.050738  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:110
05/31/2026-11:58:47.843361  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8888
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8890
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8889
ubuntu@ubuntu-server:~$

```

Rysunek 13. Przedstawienie logów po testach z nmap

```

ubuntu@ubuntu-server:~$ sudo tail -30 /var/log/suricata/fast.log
05/31/2026-11:23:43.968373  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.8:8 -> 10.0.2.15:0
05/31/2026-11:23:43.968409  [**] [1:1000001:1] ICMP Ping Detected [**] [Class
ification: (null)] [Priority: 3] {ICMP} 10.0.2.15:0 -> 10.0.2.8:0
05/31/2026-11:58:37.050484  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:143
05/31/2026-11:58:37.050485  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.
2.15:8888
05/31/2026-11:58:37.050738  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:110
05/31/2026-11:58:47.843361  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8888
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8890
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.
2.15:8889
05/31/2026-12:03:55.958821  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETEC
TED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.
2.15:8888
05/31/2026-12:03:55.958821  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:1025
05/31/2026-12:03:55.960139  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Class
ification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:5900
ubuntu@ubuntu-server:~$

```

Rysunek 14. Przedstawienie logów po testach z nmap

```

ubuntu@ubuntu-server:~$ sudo tail -20 /var/log/suricata/fast.log
[sudo: authenticate] Password:
05/31/2026-11:23:43.968373  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 10.0.2.8:8 -> 10.0.2.15:0
05/31/2026-11:23:43.968409  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 10.0.2.15:0 -> 10.0.2.8:0
05/31/2026-11:58:37.050484  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:143
05/31/2026-11:58:37.050485  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:8888
05/31/2026-11:58:37.050738  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:110
05/31/2026-11:58:47.843361  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8888
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8890
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8889
05/31/2026-12:03:55.958821  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:8888
05/31/2026-12:03:55.958821  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:1025
05/31/2026-12:03:55.960139  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:5900

```

Logi 3. Sprawdzenie reguł własnych w trybie NIDS

Który tryb pracy do tego się wykorzystuje?

Do wykrywania ataków w przeprowadzonym ćwiczeniu wykorzystano tryb NIDS. Tryb ten analizuje ruch sieciowy na podstawie zdefiniowanych reguł detekcji i generuje alerty w przypadku wykrycia podejrzanego

aktywności. W przeciwieństwie do trybu Sniffer oraz Packet Logger, które służą głównie do obserwacji i rejestrowania ruchu, tryb NIDS umożliwia automatyczne wykrywanie zagrożeń, takich jak skanowanie portów, ataki typu SYN Flood, próby logowania czy wykorzystywanie niedozwolonych metod protokołów.

W logach Suricata zarejestrowano alerty wygenerowane przez skonfigurowane reguły NIDS. Wykryto między innymi skanowanie portów (PORT SCAN DETECTED), próby połączeń z monitorowanymi portami 8888–8890 (HTTP TEST PORT 8888-8890 DETECTED) oraz zdarzenia zakwalifikowane jako potencjalny atak SYN Flood (SYN FLOOD DETECTED). Na podstawie zapisanych logów możliwe było określenie adresu IP źródłowego ataku, adresu IP hosta docelowego, numerów portów oraz rodzaju wykrytego zagrożenia. Wyniki potwierdzają poprawne działanie skonfigurowanych reguł detekcji oraz skuteczne wykrywanie aktywności generowanej podczas testów Nmap. Przykładowe wpisy z logów przedstawiają wykrycie skanowania portów hosta 10.0.2.15, prób testowania portów 8888-8890 oraz przekroczenia progu detekcji dla reguły SYN Flood. Wszystkie przeprowadzone testy zostały poprawnie rozpoznane przez system IDS.

Detekcja ataku SYN Flood

Atak SYN Flood polega na wysyłaniu dużej liczby pakietów SYN do usług sieciowych w celu przeciążenia serwera. Najczęściej atakowane są standardowe porty, np. 21 (FTP), 22 (SSH), 80 (HTTP) oraz 443 (HTTPS).

Detekcja odpowiedzi 302 serwera WWW

Kod HTTP 302 oznacza tymczasowe przekierowanie użytkownika na inny adres URL. Monitorowanie takich odpowiedzi pozwala wykrywać podejrzane przekierowania oraz nieprawidłowe działanie aplikacji webowych.

Testy skryptów Nmap NSE

W celu sprawdzenia skuteczności działania systemu NIDS wykorzystano również skrypty NSE. Przeanalizowano dostępne skrypty przeznaczone do testów typu brute-force, DoS oraz vulnerability. Następnie wykonano trzy przykładowe testy na hoście chronionym przez Suricata.

http-vuln-cve2010-2861 – skrypt sprawdzający podatność usługi HTTP,

smb-vuln-regsvcs-dos – skrypt testujący możliwość przeprowadzenia ataku typu DoS,

ssh-brute – skrypt wykonujący wielokrotne próby logowania do usługi SSH.

Podczas działania skryptów Suricata monitorowała ruch sieciowy i rejestrowała zdarzenia w plikach logów.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
└─$ nmap --script http-vuln-cve2010-2861 -p 80 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 10:19 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00082s latency).

PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds

(kali@kali)-[~]
└─$ nmap --script smb-vuln-regsvcs-dos 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 10:19 -0400
Nmap scan report for 10.0.2.15
Host is up (0.00031s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:4A:26:DA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.76 seconds
```

Rysunek 15. Wykonanie skryptów DoS oraz vulnerability

```
(kali@kali)-[~]
$ nmap --script ssh-brute -p 22 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-31 10:19 -0400
NSE: [ssh-brute] Trying username/password pair: root:root
NSE: [ssh-brute] Trying username/password pair: admin:admin
NSE: [ssh-brute] Trying username/password pair: administrator:administrator
NSE: [ssh-brute] Trying username/password pair: webadmin:webadmin
NSE: [ssh-brute] Trying username/password pair: sysadmin:sysadmin
NSE: [ssh-brute] Trying username/password pair: netadmin:netadmin
NSE: [ssh-brute] Trying username/password pair: guest:guest
NSE: [ssh-brute] Trying username/password pair: user:user
NSE: [ssh-brute] Trying username/password pair: web:web
NSE: [ssh-brute] Trying username/password pair: test:test
NSE: [ssh-brute] Trying username/password pair: root:
NSE: [ssh-brute] Trying username/password pair: administrator:
NSE: [ssh-brute] Trying username/password pair: webadmin:
NSE: [ssh-brute] Trying username/password pair: sysadmin:
NSE: [ssh-brute] Trying username/password pair: netadmin:
NSE: [ssh-brute] Trying username/password pair: guest:
NSE: [ssh-brute] Trying username/password pair: user:
NSE: [ssh-brute] Trying username/password pair: web:
NSE: [ssh-brute] Trying username/password pair: test:
NSE: [ssh-brute] Trying username/password pair: root:123456
NSE: [ssh-brute] Trying username/password pair: administrator:123456
NSE: [ssh-brute] Trying username/password pair: webadmin:123456
Stats: 0:00:34 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 10:20 (0:00:00 remaining)
```

Rysunek 16. Wykonanie skrypta brute-force

Przedstawione logi potwierdzają wykrycie aktywności wygenerowanej podczas testów Nmap NSE. Suricata zarejestrowała liczne alerty związane ze skanowaniem portów (PORT SCAN DETECTED) oraz zwiększoną liczbą pakietów SYN (SYN FLOOD DETECTED). Dodatkowo wykryto próby komunikacji z portami 8888–8890 (HTTP TEST PORT 8888-8890 DETECTED). Alerty zawierają czas zdarzenia, adres IP źródłowy i docelowy, numer portu oraz identyfikator reguły odpowiedzialnej za wykrycie zdarzenia.

```
ubunt@ubuntserver:~$ sudo tail -30 /var/log/suricata/fast.log
```

```
05/31/2026-11:58:37.050484  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:54540 -> 10.0.2.15:143
05/31/2026-11:58:37.050485  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:54540 -> 10.0.2.15:8888
05/31/2026-11:58:37.050738  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:54540 -> 10.0.2.15:110
05/31/2026-11:58:47.843361  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8888
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8890
05/31/2026-11:58:47.843362  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:46349 -> 10.0.2.15:8889
05/31/2026-12:03:55.958821  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:47516 -> 10.0.2.15:8888
05/31/2026-12:03:55.958821  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:47516 -> 10.0.2.15:1025
05/31/2026-12:03:55.960139  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:47516 -> 10.0.2.15:5900
05/31/2026-12:55:08.779270  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:43944 -> 10.0.2.15:1723
05/31/2026-12:55:08.779720  [**] [1:1000005:1] HTTP TEST PORT 8888-8890 DETECTED [**] [Classification: (null)]
[Priority: 3] {TCP} 10.0.2.8:43944 -> 10.0.2.15:8888
05/31/2026-12:55:08.779987  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:43944 -> 10.0.2.15:25
05/31/2026-12:55:25.352837  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:49626 -> 10.0.2.15:21
05/31/2026-12:55:27.733782  [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:59832 -> 10.0.2.15:21
05/31/2026-12:55:31.175260  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:59982 -> 10.0.2.15:21
05/31/2026-12:55:38.736793  [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:60400 -> 10.0.2.15:22
```

```

05/31/2026-12:55:39.627976 [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:60462 -> 10.0.2.15:22
05/31/2026-12:55:44.197684 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:60868 -> 10.0.2.15:22
05/31/2026-12:55:48.213895 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:43932 -> 10.0.2.15:22
05/31/2026-12:55:48.219156 [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:43942 -> 10.0.2.15:22
05/31/2026-12:55:53.228894 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:60758 -> 10.0.2.15:22
05/31/2026-12:55:58.232582 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:48522 -> 10.0.2.15:22
05/31/2026-12:55:58.240395 [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:48544 -> 10.0.2.15:22
05/31/2026-12:56:03.223285 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:37174 -> 10.0.2.15:22
05/31/2026-12:56:08.220154 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:48596 -> 10.0.2.15:22
05/31/2026-12:56:08.224802 [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:48606 -> 10.0.2.15:22
05/31/2026-12:56:13.221429 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:45484 -> 10.0.2.15:22
05/31/2026-12:56:18.229710 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:34006 -> 10.0.2.15:22
05/31/2026-12:56:18.237305 [**] [1:1000002:1] SYN FLOOD DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:34034 -> 10.0.2.15:22
05/31/2026-12:56:23.235048 [**] [1:1000001:1] PORT SCAN DETECTED [**] [Classification: (null)] [Priority: 3] {TCP}
10.0.2.8:41706 -> 10.0.2.15:22

```

Logi 4. Potwierdzają wykrycie aktywności podczas testów Nmap NSE zapisane w fast.log

W pliku eve.json zapisane zostały szczegółowe informacje o analizowanych połączeniach. Widoczne są wpisy dotyczące sesji SSH wygenerowanych przez skrypt ssh-brute, informacje o przepływach sieciowych (flow), wykrytych anomaliach protokołu SSH oraz statystyki działania Suricata. Dane te umożliwiają dokładniejszą analizę incydentu niż uproszczone alerty zapisane w fast.log.

```

ubunt@ubuntserver:~$ sudo tail -20 /var/log/suricata/eve.json
{"timestamp": "2026-05-31T12:57:24.177374+0000", "flow_id": "924124179136379", "in_iface": "enp0s3", "event_type": "anomaly", "src_ip": "10.0.2.8", "src_port": 37334, "dest_ip": "10.0.2.15", "dest_port": 22, "proto": "TCP", "ip_v": 4, "pkt_src": "stream", "pkt_dest": "stream", "app_proto": "ssh", "type": "aplayer", "event": "APPLAYER_DETECT_PROTOCOL_ONLY_ONE_DIRECTION", "layer": "proto_detect_timeout"}, {"anomaly": {"app_proto": "ssh", "type": "aplayer", "event": "APPLAYER_DETECT_PROTOCOL_ONLY_ONE_DIRECTION", "layer": "proto_detect_timeout"}, {"timestamp": "2026-05-31T12:57:24.177374+0000", "flow_id": "924124179136379", "in_iface": "enp0s3", "event_type": "anomaly", "src_ip": "10.0.2.8", "src_port": 37334, "dest_ip": "10.0.2.15", "dest_port": 22, "proto": "TCP", "ip_v": 4, "pkt_src": "stream", "pkt_dest": "stream", "app_proto": "ssh", "type": "aplayer", "event": "invalid_banner", "layer": "proto_parser"}}, {"timestamp": "2026-05-31T12:57:24.177374+0000", "flow_id": "924124179136379", "in_iface": "enp0s3", "event_type": "ssh", "src_ip": "10.0.2.8", "src_port": 37334, "dest_ip": "10.0.2.15", "dest_port": 22, "proto": "TCP", "ip_v": 4, "pkt_src": "stream", "pkt_dest": "stream", "app_proto": "ssh", "type": "ssh", "event": "ssh", "layer": "ssh", "client": {"proto_version": "2.0", "software_version": "libssh2_1.11.1"}}, {"timestamp": "2026-05-31T12:57:24.177449+0000", "flow_id": "924124179136379", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "10.0.2.8", "src_port": 37334, "dest_ip": "10.0.2.15", "dest_port": 22, "ip_v": 4, "proto": "TCP", "app_proto": "ssh", "flow": {"pkts_to_server": 4, "pkts_to_client": 5, "bytes_to_server": 296, "bytes_to_client": 340, "start": "2026-05-31T12:56:03.280700+0000", "end": "2026-05-31T12:56:03.281633+0000", "age": 0, "state": "established", "reason": "shutdown", "alerted": false, "exception_policy": [{"target": "app_layer_error", "policy": "ignore"}], "tx_cnt": 1, "tcp": {"tcp_flags": "1f", "tcp_flags_ts": "1a", "tcp_flags_tc": "1f", "syn": true, "fin": true, "rst": true, "psh": true, "ack": true, "state": "fin_wait1", "ts_max_regions": 1, "tc_max_regions": 1}}, {"timestamp": "2026-05-31T12:57:24.177518+0000", "flow_id": "1826104956485536", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "10.0.2.8", "src_port": 39588, "dest_ip": "10.0.2.15", "dest_port": 22, "ip_v": 4, "proto": "TCP", "app_proto": "ssh", "flow": {"pkts_to_server": 4, "pkts_to_client": 4, "bytes_to_server": 296, "bytes_to_client": 298, "start": "2026-05-31T12:56:22.621781+0000", "end": "2026-05-31T12:56:22.623462+0000", "age": 0, "state": "closed", "reason": "shutdown", "alerted": false, "exception_policy": [{"target": "app_layer_error", "policy": "ignore"}], "tx_cnt": 1, "tcp": {"tcp_flags": "1e", "tcp_flags_ts": "1a", "tcp_flags_tc": "1e", "syn": true, "rst": true, "psh": true, "ack": true, "state": "closed", "ts_max_regions": 1, "tc_max_regions": 1}}

```

```
{"timestamp":"2026-05-31T12:57:24.177544+0000","flow_id":2106834472992732,"in_iface":"enp0s3","event_type":"anomaly","src_ip":"10.0.2.8","src_port":42612,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"anomaly":{"app_proto":"ssh","type":"applayer","event":"APPLAYER_DETECT_PROTOCOL_ONLY_ONE_DIRECTION","layer":"proto_detect"}}}
{"timestamp":"2026-05-31T12:57:24.177544+0000","flow_id":2106834472992732,"in_iface":"enp0s3","event_type":"anomaly","src_ip":"10.0.2.8","src_port":42612,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"tx_id":0,"anomaly":{"app_proto":"ssh","type":"applayer","event":"invalid_banner","layer":"proto_parser"}}}
{"timestamp":"2026-05-31T12:57:24.177544+0000","flow_id":2106834472992732,"in_iface":"enp0s3","event_type":"ssh","src_ip":"10.0.2.8","src_port":42612,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"ssh":{"client":{"proto_version":"2.0","software_version":"libssh2_1.11.1"}}}
{"timestamp":"2026-05-31T12:57:24.177609+0000","flow_id":2106834472992732,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.8","src_port":42612,"dest_ip":"10.0.2.15","dest_port":22,"ip_v":4,"proto":"TCP","app_proto":"ssh","flow":{"pkts_toserver":4,"pkts_toclient":4,"bytes_toserver":296,"bytes_toclient":286,"start":"2026-05-31T12:56:23.490535+0000","end":"2026-05-31T12:56:23.492496+0000","age":0,"state":"closed","reason":"shutdown","alerted":false,"exception_policy":{"target":"app_layer_error","policy":"ignore"}},tx_cnt":1,"tcp":{"tcp_flags":"1f","tcp_flags_ts":"1a","tcp_flags_tc":"1f","syn":true,"fin":true,"rst":true,"psh":true,"ack":true,"state":"closed"},"ts_max_regions":1,"tc_max_regions":1}}
{"timestamp":"2026-05-31T12:57:24.177659+0000","flow_id":269615171698664,"in_iface":"enp0s3","event_type":"anomaly","src_ip":"10.0.2.8","src_port":49916,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"anomaly":{"app_proto":"ssh","type":"applayer","event":"APPLAYER_DETECT_PROTOCOL_ONLY_ONE_DIRECTION","layer":"proto_detect"}}}
{"timestamp":"2026-05-31T12:57:24.177659+0000","flow_id":269615171698664,"in_iface":"enp0s3","event_type":"anomaly","src_ip":"10.0.2.8","src_port":49916,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"tx_id":0,"anomaly":{"app_proto":"ssh","type":"applayer","event":"invalid_banner","layer":"proto_parser"}}}
{"timestamp":"2026-05-31T12:57:24.177659+0000","flow_id":269615171698664,"in_iface":"enp0s3","event_type":"ssh","src_ip":"10.0.2.8","src_port":49916,"dest_ip":"10.0.2.15","dest_port":22,"proto":"TCP","ip_v":4,"pkt_src":"stream" (flow timeout),"ssh":{"client":{"proto_version":"2.0","software_version":"libssh2_1.11.1"}}}
{"timestamp":"2026-05-31T12:57:24.177719+0000","flow_id":269615171698664,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.8","src_port":49916,"dest_ip":"10.0.2.15","dest_port":22,"ip_v":4,"proto":"TCP","app_proto":"ssh","flow":{"pkts_toserver":4,"pkts_toclient":5,"bytes_toserver":296,"bytes_toclient":340,"start":"2026-05-31T12:56:08.652598+0000","end":"2026-05-31T12:56:08.653335+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"exception_policy":{"target":"app_layer_error","policy":"ignore"}},tx_cnt":1,"tcp":{"tcp_flags":"1f","tcp_flags_ts":"1a","tcp_flags_tc":"1f","syn":true,"fin":true,"rst":true,"psh":true,"ack":true,"state":"fin_wait1"},"ts_max_regions":1,"tc_max_regions":1}}
{"timestamp":"2026-05-31T12:57:24.191884+0000","flow_id":1760917102803093,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":46744,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":92,"bytes_toclient":124,"start":"2026-05-31T12:55:42.344459+0000","end":"2026-05-31T12:55:42.377693+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-31T12:57:24.192056+0000","flow_id":1637718042560719,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":41798,"dest_ip":"193.25.222.136","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T12:56:29.381310+0000","end":"2026-05-31T12:56:29.437016+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T12:57:24.192074+0000","flow_id":817438839648020,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":47821,"dest_ip":"194.146.251.114","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T12:54:10.059252+0000","end":"2026-05-31T12:54:10.089513+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":1}}
{"timestamp":"2026-05-31T12:57:24.192088+0000","flow_id":1762301667686704,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":53004,"dest_ip":"192.168.0.1","dest_port":53,"ip_v":4,"proto":"UDP","app_proto":"dns","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":92,"bytes_toclient":148,"start":"2026-05-31T12:55:42.344781+0000","end":"2026-05-31T12:55:42.382772+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
{"timestamp":"2026-05-31T12:57:24.192123+0000","flow_id":322696183530481,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":40621,"dest_ip":"89.161.47.136","dest_port":123,"ip_v":4,"proto":"UDP","app_proto":"ntp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":90,"bytes_toclient":90,"start":"2026-05-31T12:56:17.468349+0000","end":"2026-05-31T12:56:17.813168+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"exception_policy":{"target":"app_layer_error","policy":"ignore"}},tx_cnt":1}}
{"timestamp":"2026-05-31T12:57:24.192150+0000","flow_id":525400762993554,"in_iface":"enp0s3","event_type":"flow","src_ip":"10.0.2.15","src_port":68,"dest_ip":"10.0.2.3","dest_port":67,"ip_v":4,"proto":"UDP","app_proto":"dhcp","flow":{"pkts_toserver":1,"pkts_toclient":1,"bytes_toserver":338,"bytes_toclient":590,"start":"2026-05-31T12:54:49.646617+0000","end":"2026-05-31T12:54:49.652594+0000","age":0,"state":"established","reason":"shutdown","alerted":false,"tx_cnt":2}}
```


Na podstawie uzyskanych logów można stwierdzić, że skrypty Nmap NSE wygenerowały ruch wykryty przez Suricata, jednak nie wszystkie zostały rozpoznane jako konkretne podatności. Skrypt http-vuln-cve2010-2861 wygenerował aktywność HTTP, która została zarejestrowana w logach, ale nie została jednoznacznie rozpoznana jako podatność CVE-2010-2861. Skrypt smb-vuln-regsvc-dos wygenerował ruch związany z testowaniem usługi SMB/DoS, jednak w logach Suricata został on widoczny głównie jako aktywność sieciowa i skanowanie, a nie jako konkretny atak DoS na SMB. Skrypt ssh-brute wygenerował wielokrotne próby komunikacji z usługą SSH, ale w zapisanych alertach nie pojawił się osobny alert o nazwie SSH brute-force. Aktywność ta została wykryta ogólnie na poziomie ruchu TCP oraz prób połączeń. Oznacza to, że Suricata poprawnie zarejestrowała ruch wygenerowany przez skrypty NSE, ale przygotowane reguły własne wykrywały przede wszystkim ogólne symptomy aktywności, takie jak skanowanie portów, zwiększona liczba pakietów SYN oraz połączenia z monitorowanymi portami. Do jednoznacznego rozpoznania konkretnych podatności lub ataków wymagane byłyby bardziej szczegółowe reguły dopasowane do danego skryptu lub podatności.

Czy wszystkie ataki udało się wykryć i rozpoznać?

Częściowo. Suricata poprawnie wykryła i zarejestrowała aktywność wygenerowaną podczas działania skryptów Nmap NSE. W logach pojawiły się alerty związane ze skanowaniem portów, zwiększoną liczbą pakietów SYN oraz próbami komunikacji z monitorowanymi portami. System poprawnie zarejestrował aktywność pochodzącą z hosta testowego oraz zapisał szczegóły połączeń w plikach fast.log i eve.json. Nie wszystkie skrypty zostały jednak rozpoznane jako konkretne podatności lub ataki o nazwach odpowiadających użytym skryptom NSE. Wykryta została przede wszystkim aktywność sieciowa generowana przez te skrypty, natomiast identyfikacja konkretnych podatności wymagałaby zastosowania bardziej szczegółowych reguł detekcji lub dodatkowych źródeł reguł, takich jak Emerging Threats lub reguły generowane z systemów Threat Intelligence.

Lista MISP

MISP jest platformą służącą do gromadzenia, wymiany i analizy informacji o zagrożeniach cyberbezpieczeństwa. System umożliwia udostępnianie wskaźników kompromitacji (IOC), takich jak złośliwe adresy IP, domeny, adresy URL, skróty plików oraz sygnatury ataków. Dane zgromadzone w MISP mogą być wykorzystywane przez systemy IDS/IPS, w tym Suricata, do automatycznego wykrywania znanych zagrożeń.

MISP oraz integracja z Snort / Suricata?

Integracja MISP z Suricata polega na generowaniu reguł detekcji na podstawie danych zgromadzonych w MISP i ich automatycznym ładowaniu do systemu IDS. Dzięki temu Suricata może wykrywać zagrożenia opisane w bazie MISP bez konieczności ręcznego tworzenia reguł.

W ramach ćwiczenia wykorzystano narzędzie suricata-update, które umożliwia pobieranie i aktualizowanie zewnętrznych źródeł reguł wykorzystywanych przez Suricata.

```
ubuntu@ubuntu-server:~$ which suricata-update
/usr/bin/suricata-update
ubuntu@ubuntu-server:~$ suricata-update --version
suricata-update version 1.3.7
ubuntu@ubuntu-server:~$
```

Rysunek 17. Sprawdzenie wersji

W celu rozszerzenia możliwości detekcyjnych systemu dodano dwa zewnętrzne źródła reguł:

Emerging Threats Open (ET Open),
Abuse.ch URLHaus.

Emerging Threats Open zawiera reguły wykrywające między innymi skanowanie sieci, próby wykorzystania podatności, złośliwe oprogramowanie oraz komunikację z serwerami Command & Control (C2).

Abuse.ch URLHaus jest bazą złośliwych adresów URL wykorzystywanych do dystrybucji malware i umożliwia wykrywanie połączeń z niebezpiecznymi domenami.

Po pobraniu reguł zostały one zapisane w katalogu wykorzystywanym przez Suricata.

```
ubuntu@ubuntu-server:~$ sudo suricata-update list-sources
31/5/2026 -- 12:12:41 - <Info> -- Using data-directory /var/lib/suricata.
31/5/2026 -- 12:12:41 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
31/5/2026 -- 12:12:41 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
31/5/2026 -- 12:12:41 - <Info> -- Found Suricata version 8.0.3 at /usr/bin/su
ricata.
31/5/2026 -- 12:12:41 - <Warning> -- Source index does not exist, will use bu
ndled one.
31/5/2026 -- 12:12:41 - <Warning> -- Please run suricata-update update-sources
.
Name: abuse.ch/feedotracker
Vendor: Abuse.ch
Summary: Abuse.ch Feodo Tracker Botnet C2 IP ruleset
License: CC0-1.0
Name: abuse.ch/sslb1-blacklist
Vendor: Abuse.ch
Summary: Abuse.ch SSL Blacklist
License: CC0-1.0
Replaces: sslbl/ssl-fp-blacklist
Name: abuse.ch/sslb1-c2
Vendor: Abuse.ch
Summary: Abuse.ch Suricata Botnet C2 IP Ruleset
License: CC0-1.0
Replaces: sslbl/ja3-fingerprints
Name: abuse.ch/sslb1-ja3
Vendor: Abuse.ch
Summary: Abuse.ch Suricata JA3 Fingerprint Ruleset
License: CC0-1.0
Replaces: sslbl/ja3-fingerprints
Name: abuse.ch/urlhaus
Vendor: Abuse.ch
Summary: Abuse.ch URLhaus Suricata Rules
License: CC0-1.0
Name: aleksibovellan/nmap
Vendor: aleksibovellan
Summary: Suricata IDS/IPS Detection Rules Against NMAP Scans
License: MIT
```

Rysunek 18. Wykonanie komendy sudo suricata-update list-sources

```
Parameters: secret-code
Subscription: https://www.secureworks.com/contact/ (Please reference CTU Co
untermeasures)
Name: stamus/lateral
Vendor: Stamus Networks
Summary: Lateral movement rules
License: GPL-3.0-only
Name: stamus/nrd-14-open
Vendor: Stamus Networks
Summary: Newly Registered Domains Open only - 14 day list, complete
License: Commercial
Parameters: secret-code
Subscription: https://www.stamus-networks.com/stamus-labs/subscribe-to-thre
at-intel-feed
Name: stamus/nrd-30-open
Vendor: Stamus Networks
Summary: Newly Registered Domains Open only - 30 day list, complete
License: Commercial
Parameters: secret-code
Subscription: https://www.stamus-networks.com/stamus-labs/subscribe-to-thre
at-intel-feed
Name: stamus/nrd-entropy-14-open
Vendor: Stamus Networks
Summary: Newly Registered Domains Open only - 14 day list, high entropy
License: Commercial
Parameters: secret-code
Subscription: https://www.stamus-networks.com/stamus-labs/subscribe-to-thre
at-intel-feed
Name: stamus/nrd-entropy-30-open
Vendor: Stamus Networks
Summary: Newly Registered Domains Open only - 30 day list, high entropy
License: Commercial
Parameters: secret-code
Subscription: https://www.stamus-networks.com/stamus-labs/subscribe-to-thre
at-intel-feed
Name: stamus/nrd-phishing-14-open
Vendor: Stamus Networks
Summary: Newly Registered Domains Open only - 14 day list, phishing
License: Commercial
Parameters: secret-code
Subscription: https://www.stamus-networks.com/stamus-labs/subscribe-to-thre
at-intel-feed
Name: stamus/nrd-phishing-30-open
```

Rysunek 19. Wykonanie komendy sudo suricata-update list-sources

Po aktywacji źródeł reguł i wykonaniu aktualizacji narzędziem suricata-update został wygenerowany plik suricata.rules zawierający reguły pochodzące z zewnętrznych źródeł informacji o zagrożeniach. Plik ten jest automatycznie ładowany przez Suricata podczas uruchamiania i może działać równolegle z własnymi regułami zapisanymi w pliku local.rules.

```
ubuntu@ubuntu-server:~$ ls -lh /var/lib/suricata/rules
total 55M
-rw-r--r-- 1 root root 3.2K May 31 12:17 classification.config
-rw-r--r-- 1 root root 1.7K May 31 11:49 local.rules
-rw-r--r-- 1 root root 78 May 31 11:45 local.rules.bak
-rw-r--r-- 1 root root 55M May 31 12:17 suricata.rules
ubuntu@ubuntu-server:~$ sudo suricata-update list-enabled-sources
31/5/2026 -- 12:22:28 - <Info> -- Using data-directory /var/lib/suricata.
31/5/2026 -- 12:22:28 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
31/5/2026 -- 12:22:28 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
31/5/2026 -- 12:22:28 - <Info> -- Found Suricata version 8.0.3 at /usr/bin/su
ricata.
Enabled sources:
- et/open
- abuse.ch/urlhaus
ubuntu@ubuntu-server:~$
```

Rysunek 20. Przedstawienie skutków aktualizacji

```

ubuntu@ubuntu-server:~$ sudo suricata-update
[sudo: authenticate] Password:
31/5/2026 -- 14:08:21 - <Info> -- Using data-directory /var/lib/suricata.
31/5/2026 -- 14:08:21 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
31/5/2026 -- 14:08:21 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
31/5/2026 -- 14:08:21 - <Info> -- Found Suricata version 8.0.3 at /usr/bin/su
ricata.
31/5/2026 -- 14:08:21 - <Info> -- Loading /etc/suricata/suricata.yaml
31/5/2026 -- 14:08:21 - <Info> -- Disabling rules for protocol pgsql
31/5/2026 -- 14:08:21 - <Info> -- Disabling rules for protocol modbus
31/5/2026 -- 14:08:21 - <Info> -- Disabling rules for protocol dnp3
31/5/2026 -- 14:08:21 - <Info> -- Disabling rules for protocol enip
31/5/2026 -- 14:08:21 - <Info> -- Fetching https://urlhaus.abuse.ch/downloads
/urlhaus_suricata.tar.gz.
100% - 731913/731913
31/5/2026 -- 14:08:22 - <Info> -- Done.
31/5/2026 -- 14:08:22 - <Info> -- Checking https://rules.emergingthreats.net/
open/suricata-8.0.3/emerging.rules.tar.gz.md5.

```

Rysunek 21. Wykonania komend suricata-update

```

ubuntu@ubuntu-server:~$ sudo suricata-update enable-source et/open
31/5/2026 -- 12:24:52 - <Info> -- Using data-directory /var/lib/suricata.
31/5/2026 -- 12:24:52 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
31/5/2026 -- 12:24:52 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
31/5/2026 -- 12:24:52 - <Info> -- Found Suricata version 8.0.3 at /usr/bin/su
ricata.
31/5/2026 -- 12:24:52 - <Warning> -- The source et/open is already enabled.
31/5/2026 -- 12:24:52 - <Info> -- Source et/open enabled
ubuntu@ubuntu-server:~$ sudo suricata-update enable-source abuse.ch/urlhaus
Command 'udo' not found, but can be installed with:
sudo apt install udo
ubuntu@ubuntu-server:~$ sudo suricata-update enable-source abuse.ch/urlhaus
31/5/2026 -- 12:25:10 - <Info> -- Using data-directory /var/lib/suricata.
31/5/2026 -- 12:25:10 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
31/5/2026 -- 12:25:10 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
31/5/2026 -- 12:25:10 - <Info> -- Found Suricata version 8.0.3 at /usr/bin/su
ricata.
31/5/2026 -- 12:25:10 - <Warning> -- The source abuse.ch/urlhaus is already e
nabled.
31/5/2026 -- 12:25:10 - <Info> -- Source abuse.ch/urlhaus enabled
ubuntu@ubuntu-server:~$

```

Rysunek 22. Wykonania komend suricata-update

Gotowe listy detekcji dostępne w Internecie

Emerging Threats Open (ET Open)

Jeden z najpopularniejszych publicznych zbiorów reguł dla Suricata i Snort. Zawiera reguły umożliwiające wykrywanie skanowania sieci, exploitów, malware, komunikacji C2 oraz wielu innych znanych zagrożeń.

Abuse.ch URLHaus

Zbiór reguł tworzony na podstawie bazy złośliwych adresów URL. Pozwala wykrywać komunikację z domenami wykorzystywanymi do dystrybucji malware oraz innych zagrożeń internetowych.

Wnioski

Przy wykonaniu danego ćwiczenia skonfigurowano i przetestowano system Suricata w trybach Sniffer, Packet Logger oraz NIDS. Tryb Sniffer umożliwił monitorowanie ruchu sieciowego w czasie rzeczywistym, natomiast Packet Logger pozwolił na zapis pakietów do plików PCAP i ich późniejszą analizę. W trybie NIDS wykorzystano własne reguły detekcji, które umożliwiły wykrywanie skanowania portów, prób ataków SYN Flood oraz innych podejrzanych zdarzeń sieciowych. Testy przeprowadzone przy użyciu programu Nmap oraz skryptów NSE potwierdziły skuteczność działania przygotowanych reguł. Przeprowadzone ćwiczenie pokazało, że skuteczność systemu IDS zależy przede wszystkim od poprawnej konfiguracji reguł, aktualnych baz sygnatur oraz odpowiedniego monitorowania logów. Suricata umożliwia analizę ruchu na poziomie warstw sieciowych i aplikacyjnych, a także wykrywanie zagrożeń na podstawie zawartości pakietów i zachowania ruchu sieciowego.